LinkVault - Interview Prep Guide

Your project, explained simply. Everything here is based on the real code in the `CODEIO` repo.

1. Project in 30 Seconds

What it does: LinkVault turns a long link (like a 200-character Amazon URL) into a short one (like `yoursite.com/aB3x`). When someone opens the short link, they are sent to the long one. It also counts clicks and shows charts.

Who uses it: Anyone sharing links - students, clubs, small businesses, social media posts.

The analogy (use this in the interview): It works like a **coat check at a party**. You give your big coat (the long URL) to the counter, and they give you a small token with a number (the short code). Later, anyone who shows that token gets the coat back (gets redirected to the long URL). The counter also notes how many times each token was used (click count).

3 lines to open the interview:

1. "LinkVault is a full-stack URL shortener I built with React, Node.js, Express and MongoDB."
2. "You paste a long link, get a short 4-character code or a custom name, and when someone opens it, the server looks up the code and redirects them."
3. "It also has click analytics, links that expire, and optional passcode protection."

Main features:

- **Short links** - random 4-character code, for example `/aB3x`.
- **Custom alias** - choose your own name, for example `/my-portfolio`.
- **Expiry** - a link can stop working after 1, 7 or 30 days.
- **Passcode protection** - visitors must type a passcode before being redirected.
- **Click analytics** - total clicks, last 7 days chart, top browsers, devices and referrers.
- **Dashboard** - search, sort, copy, edit, view stats and delete your links.
- **Rate limiting** - one IP can create at most 30 links every 15 minutes.
- **Fun UI** - glass-style design, animated background lines, walking Pokemon at the bottom.

2. Tech Stack

Technology	What it is	Why used here
React 18	JavaScript library for building UI	Builds all pages (Home, Dashboard, Analytics)
Vite	Fast dev server and build tool	Runs the frontend locally on port 5173 and builds it for Vercel
React Router	Page navigation without reloading	Maps URLs like `/dashboard` to page components
Axios	Library to send HTTP requests	Frontend uses it to call the backend API
Recharts	Chart library for React	Draws the 7-day bar chart and donut charts
Node.js	Runs JavaScript on the server	Runs the backend
Express 5	Web framework for Node.js	Creates API routes like `POST /api/urls`
MongoDB	NoSQL database (stores JSON-like documents)	Stores links and click records
Mongoose	Library to talk to MongoDB	Defines the `Url` and `Click` models
nanoid	Random ID generator	Creates the 4-character short code
validator	Input-checking library	Checks that the URL is valid and starts with http/https
helmet, cors, express-rate-limit	Security middleware	Safe headers, control which sites can call the API, stop spam
ua-parser-js	Reads the browser's "user agent"	Finds browser, OS and device for analytics
Jest + Supertest	Testing tools	Automated tests in `server/tests/api.test.js`

Technology	What it is	Why used here
Vercel	Hosting for frontends	Hosts the React site
Render	Hosting for backends	Planned host for the Express server (`render.yaml`)
MongoDB Atlas	Cloud database	Planned production database

3. Project Structure

```
CODEIO/
  client/                    -> Frontend (React + Vite)
    src/main.jsx             -> Starts React, wraps app in BrowserRouter
    src/App.jsx              -> All page routes + Navbar + background
    src/api/client.js        -> ALL calls to the backend (+ offline fallback)
    src/pages/Home.jsx       -> Shorten form and result card
    src/pages/Dashboard.jsx  -> List, search, edit, delete links
    src/pages/Analytics.jsx  -> Charts for one link
    src/pages/RedirectHandler.jsx -> Opens short links inside the React app
    src/components/          -> Navbar, BackgroundPaths, PokemonWalker
    vercel.json              -> Sends every URL to index.html (for routing)
  server/                    -> Backend (Node + Express)
    src/server.js            -> Starts the server and connects the DB
    src/app.js               -> Middleware + routes setup
    src/routes/              -> urlRoutes.js, redirectRoutes.js
    src/controllers/         -> urlController.js, redirectController.js (main logic)
    src/models/              -> Url.js, Click.js (database shape)
    src/middleware/          -> rateLimiter.js, errorHandler.js
    src/config/db.js         -> Connects to MongoDB
    tests/api.test.js        -> 8 automated API tests
  render.yaml                -> Deploy settings for Render
  docs/                      -> Architecture notes
```

4. How the Frontend Works

Main pages (defined in `client/src/App.jsx`):

URL	Component	What it shows
`/`	`Home.jsx`	Shorten form, live total links/clicks, result card
`/dashboard`	`Dashboard.jsx`	All links with Copy, Open, Edit, Stats, Del buttons
`/analytics/:id`	`Analytics.jsx`	Click charts for one link
`/:shortCode`	`RedirectHandler.jsx`	Opens a short link (or asks for passcode)
anything else	`NotFound.jsx`	404 page

What happens when you click "Shorten URL":

1. The form calls `handleSubmit` in `Home.jsx`.
2. It checks the URL box is not empty. If empty, shows an error toast.
3. It calls `createShortUrl({ originalUrl, customAlias, expiresInDays, passcode })` from `src/api/client.js`.
4. `createShortUrl` sends `POST /api/urls` to the backend using Axios (timeout 4 seconds).
5. The backend replies with `shortCode`, `shortUrl`, etc.
6. `Home.jsx` saves it with `setResult(data)`, shows "Short link generated successfully!", clears the form and refreshes the stats.
7. The result card appears with Copy and Open buttons and a "% Shorter" badge.

How the request reaches the backend: `client.js` creates one Axios instance with `baseURL = VITE_API_URL`. Every page uses functions from this file: `createShortUrl`, `fetchUrls`, `updateShortUrl`, `deleteShortUrl`, `fetchAnalytics`.

> **Honest note - the "fallback engine":** `client.js` has a backup plan. If `VITE_API_URL` is empty, or the backend call fails, it saves links in the browser's `localStorage` instead. In `client/.env.production`, `VITE_API_URL` is **empty**, so the live Vercel site never talks to the backend. That means a short link only works in the browser that created it. Be ready to explain this (see Q&A).

5. How the Backend Works

All API endpoints (`server/src/app.js` + `routes/`):

Method	URL	What it receives	What it returns
GET	`/api/health`	Nothing	`{ status: "ok" }`
POST	`/api/urls`	`originalUrl`, optional `customAlias`, `expiresInDays`, `passcode`	201 + `_id`, `shortCode`, `shortUrl`, `originalUrl`, `expiresAt`, `isProtected`
GET	`/api/urls`	Nothing	List of all links (newest first) with `status` active/expired. Passcode is hidden
PATCH	`/api/urls/:id`	Any of `originalUrl`, `expiresInDays`, `passcode`	Updated link
DELETE	`/api/urls/:id`	Link id in URL	Success message (also deletes its clicks)
GET	`/api/urls/:id/analytics`	Link id in URL	Total clicks, `clicksPerDay` (7 days), `topBrowsers`, `topDevices`, `topReferrers`
GET	`/:shortCode`	Short code (and `?passcode=` if locked)	302 redirect, or 404 / 410 / 401 passcode page

Request-to-response flow:

```
Browser sends request
        |
        v
server.js  (server listening on PORT, DB connected)
        |
        v
app.js middleware, in order:
   helmet()        -> adds security headers
   cors()          -> is this website allowed to call me?
   express.json()  -> turns the JSON body into req.body
        |
        v
Router decides:  /api/urls...  -> urlRoutes.js
                 /:shortCode   -> redirectRoutes.js
        |
        v
(POST only) shortenLimiter -> max 30 per 15 min per IP
        |
        v
Controller function (e.g. createUrl) -> validate -> talk to MongoDB
        |
        v
res.status(...).json(...)  or  res.redirect(302, ...)
        |
   if an error is thrown -> errorHandler.js sends a JSON error
```

Important detail: `/api` routes are mounted **before** `/:shortCode`. Otherwise `/api` itself would be treated as a short code.

6. Database

Database: MongoDB (via Mongoose). `server/src/config/db.js` tries, in order:

1. A cloud database (Atlas) if `MONGO_URI` / `MONGODB_URI` is set and is not localhost.
2. A local MongoDB on your laptop.
3. If neither works, an **in-memory** MongoDB (`mongodb-memory-server`). Easy for testing, but all data is lost when the server restarts.

Collection 1: `urls` (model in `models/Url.js`) - one document per short link:

Field	Meaning
`_id`	Unique ID MongoDB gives every document
`originalUrl`	The long link the user pasted (required)
`shortCode`	The short part, e.g. `aB3x`. **Unique + indexed** so lookups are fast and no two are the same
`clicks`	How many times the link was opened (starts at 0)
`expiresAt`	Date when the link stops working, or `null` = never

Field	Meaning
`passcode`	Optional passcode, or `null`. Stored as plain text (see weak points)
`createdAt`, `updatedAt`	Added automatically by `timestamps: true`

Collection 2: `clicks` (model in `models/Click.js`) - one document per visit:

Field	Meaning
`urlId`	Which link was clicked (points to a `urls` document)
`timestamp`	When it was clicked
`browser`, `os`, `device`	Read from the visitor's user agent, e.g. Chrome, macOS, Desktop
`referrer`	Website the visitor came from, or "Direct"

One link has many clicks. This is called a **one-to-many** relationship.

7. How Link Shortening Works (MOST IMPORTANT)

Step by step: long URL to short URL

1. User pastes `https://www.amazon.in/some/very/long/product/link?ref=abc` on the Home page and clicks **Shorten URL**.
2. Frontend sends `POST /api/urls` with `{ originalUrl: "https://..." }`.
3. `shortenLimiter` checks the IP has not created more than 30 links in 15 minutes.
4. `createUrl` in `urlController.js` runs:
   - **Validate:** URL must exist and pass `validator.isURL(...)` with `http` or `https`. Otherwise **400**.
   - **Custom alias?** If given: must be 3-20 characters, only letters, numbers, `-` or `_`, not a reserved word (`api`, `health`, `dashboard`, `analytics`, `docs`, `favicon.ico`), and not already used (**409** if taken).
   - **No alias?** Generate a random 4-character code (below).
   - **Expiry:** if `expiresInDays` is 7, `expiresAt = now + 7 days`.
   - **Save** with `Url.create({ originalUrl, shortCode, expiresAt, passcode })`.
5. Builds `shortUrl` = `BASE_URL + "/" + shortCode` (function `buildShortUrl`).
6. Returns **201 Created** with the short URL. The frontend shows it.

How the short code is generated (your actual code)

From `server/src/controllers/urlController.js`:

```
let isUnique = false;
let retries = 0;
const MAX_RETRIES = 5;

while (!isUnique && retries < MAX_RETRIES) {
  shortCode = nanoid(4);
  const existing = await Url.findOne({ shortCode });
  if (!existing && !RESERVED_ALIASES.has(shortCode.toLowerCase())) {
    isUnique = true;
  }
  retries++;
}
```

- `nanoid(4)` picks 4 random characters from 64 options (`A-Z`, `a-z`, `0-9`, `_`, `-`).
- Total combinations: 64 x 64 x 64 x 64 = **16,777,216** (about 16.7 million).
- **Real example:** `nanoid(4)` returns `aB3x`. The database has no `aB3x`, so it is saved. Short URL = `http://localhost:5001/aB3x`.

How duplicates / collisions are handled

A **collision** means the random code already exists.

- **Random codes:** check the DB with `Url.findOne({ shortCode })`. If taken, try again - up to **5 times**. If all 5 fail, return **500** "Could not generate a unique short code".

- **Custom alias:** if taken, return **409 Conflict** ("Custom alias is already in use").
- **Safety net:** `shortCode` has `unique: true` in `Url.js`, so MongoDB itself refuses duplicates even if two requests arrive at the same moment.
- Note: the same long URL shortened twice gets **two different** short codes. The code does not reuse old ones.

What happens when someone opens the short link

`GET /:shortCode` -> `handleRedirect` in `redirectController.js`:

1. **Lookup:** `Url.findOne({ shortCode })`. Not found -> **404**.
2. **Expired?** If `expiresAt` is in the past -> **410 Gone**.
3. **Passcode?** If the link has a passcode and the visitor did not send the right one (`?passcode=...`) -> **401** + a small HTML page asking for it.
4. **Count the click:** `recordClickAsync(url._id, req)` is called **without** `await`. It increases `clicks` and saves a `Click` record in the background, so the visitor does not wait.
5. **Redirect:** `res.redirect(302, url.originalUrl)`.

301 vs 302 in simple words

- **301 = "moved permanently".** The browser remembers it and next time goes straight to the long URL **without asking your server**. You would miss clicks, and you could not change the destination later.
- **302 = "moved temporarily".** The browser asks your server every time. Your code uses **302**, so every click is counted and links can be edited or expired.

Extra features

- **Click count:** `clicks` field + one `Click` document per visit.
- **Analytics:** `getAnalytics` runs 4 MongoDB **aggregation** queries in parallel (`Promise.all`): clicks per day, top 5 browsers, devices, referrers. Missing days are filled with 0.
- **Expiry:** `expiresAt` + 410 response. Dashboard shows ACTIVE / EXPIRING SOON (under 24h) / EXPIRED.
- **Edit:** `PATCH /api/urls/:id` changes destination, expiry or passcode without deleting the link.

8. API Keys, Environment Variables and Config

Good news: your project uses **no paid API keys**. Only environment variables and a few external services.

Server (`server/.env`, template in `server/.env.example`):

Variable	What it does	Where used	If missing
`PORT`	Port the server listens on	`server.js`	Uses 5001 by default
`MONGO_URI` / `MONGODB_URI`	Address of the MongoDB database (Atlas contains username + password, so it is a **secret**)	`config/db.js`	Falls back to local or in-memory DB. Data disappears on restart. On Render it may fail completely
`BASE_URL`	Domain used to build `shortUrl`	`buildShortUrl` in `urlController.js`	Uses the request's own host - usually still works
`CLIENT_URL`	Frontend address allowed by CORS	`app.js`	In production, only `*.vercel.app` and localhost are allowed. A custom domain gets blocked
`NODE_ENV`	Says "production" or "development"	`app.js`, `errorHandler.js`	If not `production`, CORS allows every website

Client (`client/.env`, `client/.env.production`):

Variable	What it does	Where used	If missing / empty
`VITE_API_URL`	Backend address for Axios	`src/api/client.js`	App switches to browser-only `localStorage` mode. **This is the current state on Vercel**

External services: MongoDB Atlas (database), Render (backend host), Vercel (frontend host), Google Fonts (`index.html`), Pokemon Showdown image links (`PokemonWalker.jsx`).

What is a `.env` file? A small text file of `NAME=value` settings that live **outside the code**. Like keeping your house key in your pocket instead of taping it to the front door. The code reads them with `process.env.NAME` (backend) or `import.meta.env.VITE_NAME` (frontend).

Why never push it to GitHub? Anyone can read a public repo. If your `MONGO_URI` with password leaks, a stranger can read or delete your whole database. Your `.gitignore` already blocks `.env` files - good.

> **Honest notes:** (1) Anything starting with `VITE_` is put inside the frontend JavaScript, so it is **public** - never put secrets there. (2) `LinkVault_Project_Full.tar.gz` is committed to GitHub and contains copies of both `.env` files. I checked: they only hold localhost values, so nothing secret leaked - but it should be removed.

9. Technical Terms Glossary

Term	One-line meaning	Where in your project
API	A way for two programs to talk using requests and responses	Your Express backend
REST	A style where URLs are things and HTTP methods are actions	`/api/urls` with GET, POST, PATCH, DELETE
Endpoint	One specific URL + method of the API	`POST /api/urls`
HTTP methods	GET read, POST create, PATCH update, DELETE remove	`urlRoutes.js`
Status code	Number that tells the result: 200 OK, 201 Created, 400 bad input, 404 not found	Every controller
Redirect (302)	Server tells the browser "go to this other URL"	`res.redirect(302, ...)`
Middleware	Function that runs before the main handler	`helmet`, `cors`, `shortenLimiter`, `errorHandler`
CORS	Browser rule deciding which websites may call your API	`cors({...})` in `app.js`
Rate limiting	Limit how many requests one user can make	`rateLimiter.js` (30 / 15 min)
Database / NoSQL	Stores data as JSON-like documents, not tables	MongoDB
Schema / Model	The shape of the data	`Url.js`, `Click.js`
Index	Makes searching a field fast	`shortCode` (`index: true, unique: true`)
Aggregation	Grouping and counting data inside MongoDB	`Click.aggregate` in `getAnalytics`
Async / await	Wait for slow work (like DB) without freezing	Every controller
Hashing	Turning a password into scrambled text that cannot be reversed	**Not used** - passcodes are plain text (an improvement to mention)
Environment variable	A setting stored outside the code	`.env` files
SPA	Single Page App - one HTML page, React swaps the content	React + `vercel.json` rewrite
localStorage	Small storage inside the user's browser	Fallback engine in `client.js`
Deployment	Putting the app on the internet	Vercel + Render
Unit / integration test	Code that checks your code automatically	`api.test.js`

10. Easy Interview Q&A

1. What is your project? LinkVault is a URL shortener. You paste a long link and get a short one, and when someone opens the short link the server redirects them. It also tracks clicks, supports custom names, expiry and passcodes.

2. Why did you build it? A URL shortener looks simple but covers the full stack - a frontend form, a REST API, a database and redirects. It was a good way to learn how all the parts connect.

3. What tech stack did you use and why? I used the MERN stack: MongoDB, Express, React and Node. All of it is JavaScript, so I use one language on both sides. MongoDB fits well because each link is just a small JSON document.

4. How does shortening work? The frontend sends the long URL to `POST /api/urls`. The backend validates it, makes a random 4-character code with `nanoid`, checks the database that the code is free, saves it and returns the short URL.

5. How do you make sure two links don't get the same code? Before saving, I check with `Url.findOne`. If the code is taken, I retry up to 5 times. Also, the `shortCode` field is marked unique in MongoDB, so the database blocks duplicates anyway.

6. Why only 4 characters? Isn't that too small? With 64 possible characters, 4 characters give about 16.7 million combinations - plenty for a small project. If it grows, I would just increase the length to 6 or 7.

7. What happens when someone opens a short link? The server looks up the code. If it's missing I send 404, if expired 410, and if locked a passcode page. Otherwise I record the click in the background and send a 302 redirect to the original URL.

8. Why 302 and not 301? A 301 is cached by the browser, so later clicks never reach my server and I can't count them. A 302 makes the browser ask every time, so analytics stay accurate and I can still edit or expire the link.

9. What is an API? An API is a set of rules for programs to talk. My React app sends HTTP requests to my Express API, and it replies with JSON.

10. What is REST? REST is a style of designing APIs where the URL is the resource and the HTTP method is the action. For example, GET `/api/urls` lists links and DELETE `/api/urls/:id` deletes one.

11. What database did you use and what does it store? MongoDB. The `urls` collection stores the original URL, short code, click count, expiry and passcode. The `clicks` collection stores one record per visit with browser, device and referrer.

12. What is middleware? Middleware is a function that runs before the main route handler, like a security check at a gate. I use `helmet` for security headers, `cors`, a rate limiter, and a global error handler.

13. What is CORS? Browsers block a website from calling another domain's API unless that API allows it. My `app.js` allows localhost, my `CLIENT_URL`, and Vercel domains.

14. How do the analytics work? On every redirect I save a `Click` with browser and device from `ua-parser-js`. The analytics endpoint uses MongoDB aggregation to count clicks per day and find the top browsers, devices and referrers.

15. Why is click tracking not awaited? So the redirect stays fast. The visitor is sent on immediately, and the click is saved in the background. If saving fails, the user is not affected.

16. How do you protect the server from spam? I use `express-rate-limit` on link creation - 30 links per 15 minutes per IP. I also validate every URL and alias before saving.

17. Where is it deployed? The frontend is on Vercel. I prepared `render.yaml` to deploy the backend on Render with MongoDB Atlas. Right now the live site runs in a browser-only mode, which I plan to connect to the real backend.

18. Did you write tests? Yes, `server/tests/api.test.js` has 8 tests with Jest and Supertest. They use an in-memory MongoDB and check creating links, invalid URLs, editing, redirects and passcodes.

19. What challenges did you face? Making the backend always available on free hosting was hard, so a fallback was added that stores links in the browser. I learned that this fallback has a trade-off: links only work in the creator's browser. Ordering routes was another one - `/api` must come before `/:shortCode`.

20. What would you improve? First, connect the live site to the real backend so shared links work for everyone. Then hash passcodes with bcrypt, add user login so each person sees only their own links, and add a cache like Redis for faster lookups.

21. Is your project secure? It has good basics: helmet, CORS, input validation, rate limiting, and secrets in `.env`. But passcodes are stored as plain text and there is no login, so anyone can list all links - I would fix those next.

22. Did you use AI tools to build this? Yes, I used an AI coding tool called Antigravity to speed up writing the code, the same way many developers now use Copilot or Cursor. But I made the decisions on features, tested it, and I've studied the code so I can explain how each part works. I also found some issues in it myself, which taught me that AI code still needs a human to review it.

23. What did you learn? I learned how a request travels from a React form, through Express middleware and routes, to MongoDB and back. I also learned about HTTP status codes, redirects, and why secrets stay in `.env` files.

11. "What If I Don't Know?" Guide

Ready-to-use sentences:

1. "I'm not 100% sure about that, but here's how I think it works..."

2. "I haven't gone deep into that part yet. What I do know is..."
3. "That's a good question. In my project I handled something similar in `urlController.js` by..."
4. "I don't know the exact answer, but I'd find out by reading the docs or testing it with a small example."
5. "I used a library for that - `nanoid` / `helmet` - so I know what it does, though not every detail inside it."
6. "Can I think out loud for a second?"
7. "I know that's a weak point in my project. My plan to fix it is..."
8. "I'd love to learn that properly - can you tell me how you'd approach it?"

How to give a partial answer:

- **Say what you DO know first.** Example: asked about database indexing - "I know `shortCode` has `index: true` so lookups are fast. I'm not sure how MongoDB builds the index internally."
- **Connect it to your code.** Always bring it back to a real file or function name. It shows you actually read your project.
- **Show willingness to learn.** End with what you would do next: "I'd read up on that and try it in LinkVault."

What NOT to say:

- "The AI wrote that part, I don't know." - Say instead: "I'm still studying that part, here's what I understand."
- "I don't know." and then silence. Always add a guess or what you would do.
- Don't invent features you don't have (e.g. "I used Redis" - you didn't).
- Don't blame tools or hide problems. Admitting a known bug + a fix plan looks strong.
- Don't use big words you can't explain - the next question will be "what does that mean?"

Weak Points You Should Know (Honest Review)

Interviewers love asking "what's wrong with your project?" Knowing these makes you look strong.

Problem	Where	Simple explanation
Live site doesn't use the backend	`client/.env.production` (`VITE_API_URL` empty)	Links are saved only in the creator's browser. A friend opening your short link sees "Link Not Found"
Analytics page likely crashes on the live site	`fetchAnalytics` in `client.js` vs `Analytics.jsx`	The fallback returns data in a different shape (`analytics.browsers`) than the page expects (`topBrowsers`)
Fake analytics in fallback mode	`fetchAnalytics` in `client.js`	Charts are estimated from the click count using fixed percentages, not real data
Passcodes not hashed	`Url.js`, `client.js`	Stored as plain text. Should use `bcrypt`
No user login	whole app	`GET /api/urls` returns every link to everyone
Error handler bug	`errorHandler.js`	Missing brackets around `err.statusCode || ...` can send the wrong status code
Old docs	`docs/SYSTEM-ARCHITECTURE.md`	Still says `nanoid(7)` and `/stats`; code uses `nanoid(4)` and `/analytics`

12. One-Page Cheat Sheet

1-minute pitch (memorize):

"LinkVault is a full-stack URL shortener built with the MERN stack - MongoDB, Express, React and Node. A user pastes a long URL on the React frontend, which sends a POST request to my Express API. The backend validates the URL, generates a random 4-character code with nanoid, checks MongoDB to avoid duplicates, and saves it. When someone opens the short link, the server looks up the code and sends a 302 redirect, while recording the click in the background for analytics. On top of that, I added custom aliases, expiring links, passcode protection, a dashboard with charts, and rate limiting for security. The frontend is on Vercel, and next I want to connect the live site fully to the backend and add user accounts."

Quick revision points:

- **Stack:** React + Vite (frontend), Node + Express (backend), MongoDB + Mongoose (database), Vercel + Render (hosting).
- **Main logic file:** `server/src/controllers/urlController.js` -> `createUrl`.
- **Redirect file:** `server/src/controllers/redirectController.js` -> `handleRedirect`.
- **Frontend API file:** `client/src/api/client.js` -> `createShortUrl`.
- **Short code:** `nanoid(4)` -> 64^4 = about 16.7 million combos. Retry 5 times on collision. `shortCode` is unique + indexed.
- **Alias rules:** 3-20 chars, letters/numbers/`-`/_, not reserved, 409 if taken.
- **Redirect:** found -> 302. Not found 404. Expired 410. Locked 401.
- **302 not 301:** so every click reaches the server and gets counted.
- **Click tracking:** `recordClickAsync` without `await` = fast redirect.
- **Models:** `Url` (originalUrl, shortCode, clicks, expiresAt, passcode) and `Click` (urlId, timestamp, browser, os, device, referrer).
- **Middleware order:** helmet -> cors -> express.json -> routes -> 404 -> errorHandler.
- **Security:** helmet, CORS, validator, rate limit 30 / 15 min, `.env` in `.gitignore`.
- **Env vars:** `MONGO_URI`, `PORT`, `BASE_URL`, `CLIENT_URL`, `NODE_ENV`, `VITE_API_URL`.
- **Tests:** 8 tests, Jest + Supertest, in-memory MongoDB.
- **Honest weak points:** live site in browser-only mode, plain-text passcodes, no login.
- **AI answer:** "Yes, I used Antigravity to speed up coding, but I reviewed, tested and studied every part."